

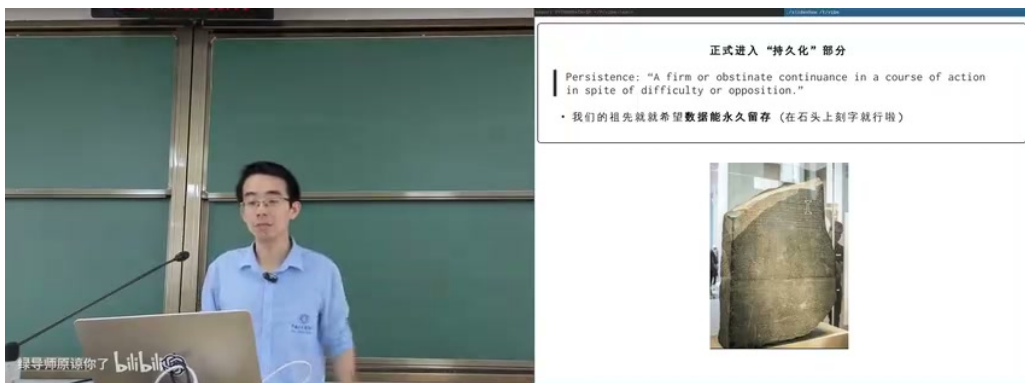
课程笔记

23 - 存储设备原理

2026 南京大学操作系统原理

lecture-to-notes

2026 年



视频作者/频道：南京大学操作系统原理（蒋炎岩）

发布日期：2026

视频时长：01:19:30

视频链接：本地课程视频

目录

1 从持久化到存储设备抽象	2
1.1 存储设备的共同接口	3
1.2 本章小结	3
2 磁：从一维磁带到二维磁盘	3
2.1 磁带：密度很高，随机访问很差	3
2.2 磁鼓：牺牲容量换定位延迟	4
2.3 硬盘：旋转、寻道和工程极限	5
2.4 本章小结	7
3 可移动介质：软盘与光盘	7
3.1 软盘：低成本介质与数据交换文化	7
3.2 光盘：把坑做成数字内容	7
3.3 本章小结	8
4 只追加写与 Project Silica	8
4.1 Random Read + Append-only Write	8
4.2 Project Silica：在玻璃里写入长期数据	9
4.3 本章小结	9
5 Flash 与 SSD：电路速度下的持久化	10
5.1 Floating Gate：用电荷保存 bit	10
5.2 磨损、映射表和 FTL	10
5.3 本章小结	11
6 块设备：操作系统看到的统一形状	12
6.1 为什么是 block array	12
6.2 读放大、写放大和局部性	12
6.3 把 OS 原理做成可观察工具	13
6.4 本章小结	13
7 总结与延伸	14

1 从持久化到存储设备抽象

这一讲从上一讲的设备模型自然过渡到持久化。上一讲的核心模型是 canonical device：设备通过状态、数据、命令等寄存器与 CPU 交互，Linux 再用 `struct file_operations` 或相邻的驱动接口把设备伪装成文件。本讲关心的是其中最重要的一类设备：存储设备。对操作系统来说，磁盘、SSD、U 盘、SD 卡、软盘、光盘最终都要被抽象成可读写的数据空间；但对硬件来说，一个 bit 可以通过磁化方向、坑、光学状态、电荷、晶体微结构等完全不同的物理机制存在。

核心问题

存储设备设计的本质不是“能否保存一个 bit”，而是同时回答三个问题：

1. 如何让一个状态在断电、搬运、震动、时间流逝后仍然存在？
2. 如何用电路或机械装置读写这个状态？
3. 如何在巨大数量的 bit 中定位目标 bit，并把定位代价控制在可接受范围内？

持久化并不神秘。石碑、甲骨、罗塞塔石都能跨越很长时间保存信息；磁性画板也能保存一段时间的图案。计算机存储设备的特殊之处在于，它必须把“可观察的状态”变成“可由电路自动读写的状态”，并且把这种状态高密度复制数以亿计、万亿计。



图 1：磁性画板展示了一个最朴素的持久化模型：磁粉在上表面或下表面形成可反复改写的状态。¹

如果把磁性画板的每个格子理解成一个 bit，上下两个位置就可以表示 0 和 1。但真正的计算机存储器还要求它可以被电信号驱动：写入时能确定地改变状态，读取时能把状态转换成电信号。于是，存储设备的历史就是把“可持久的物理状态”不断压缩、并行化、电子化的历史。

¹ 视频画面时间区间：00:05:35–00:05:40。

1.1 存储设备的共同接口

从 OS 视角看，一个存储设备常常被看成线性的 byte array。但这个视角是上层软件的便利抽象，设备本身通常并不支持任意字节粒度的单独读写。硬盘按扇区读写，SSD 按 page 读写、按 block 擦除，光盘按轨道和扇区读取，磁带更接近顺序流。

“byte array” 是软件赠予硬件的幻觉

用户程序可以 mmap 一个大文件，仿佛整个磁盘被映射成地址空间里的字节数组。但在设备层，哪怕只写一个字节，也常常需要读出一整块、修改内存中的副本、再写回一整块。这就是读放大和写放大的根源之一。

1.2 本章小结

本讲的主线是：先追问一个 bit 如何持久化，再追问许多 bit 如何高密度排列，最后追问操作系统如何把复杂设备统一成块设备抽象。理解这条线，后面的文件系统、缓存、日志、崩溃一致性都会更容易。

2 磁：从一维磁带到二维磁盘

磁存储的核心机制是利用可控的磁化方向保存 0/1。写入时，读写头用外加磁场改变局部磁化方向；读取时，介质经过读头产生可检测的电磁信号。这个想法简单到可以用“许多小磁针排成队”来理解，但工程上会被压缩到纳米尺度。

2.1 磁带：密度很高，随机访问很差

磁带可以理解成一维存储介质：bit 沿着很长的带子顺序排列。现代磁带不是粗糙的“小磁针”，而是纳米级磁性材料构成的稳定结构。它的优势非常清楚：便宜、容量大、可靠性高、顺序读写并不慢；它的短板也同样清楚：随机访问需要机械倒带或寻带，延迟巨大。

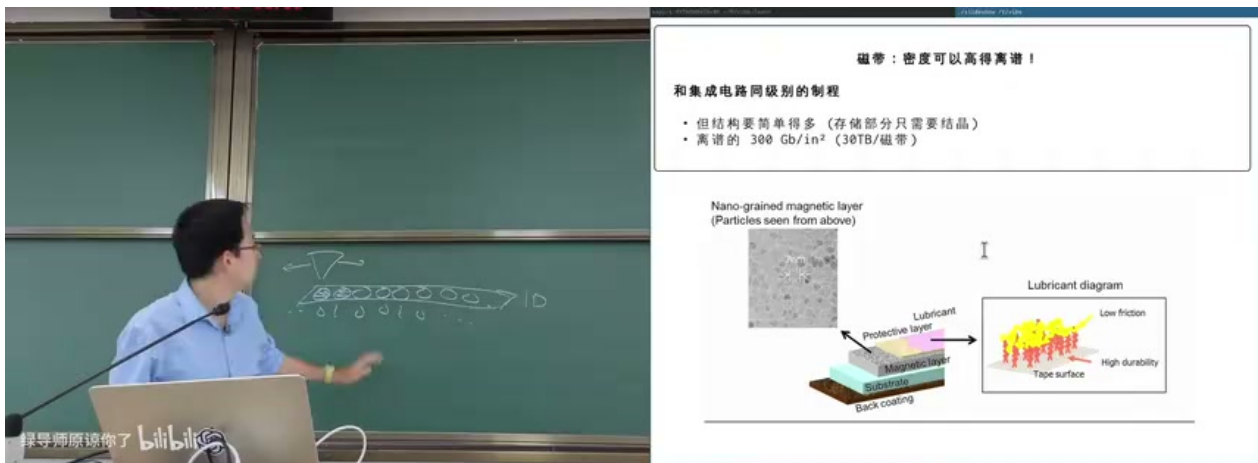


图 2: 磁带通过纳米级磁性层和润滑/保护结构实现很高的面密度，但其线性形态天然偏向顺序访问。²

讲者强调，磁带并不是“性能低”的同义词。如果访问模式是顺序的，磁带可以有很好的带宽；如果访问模式是随机的，哪怕只读一个远处的字节，也可能要移动很长一段介质。于是磁带适合冷数据归档、长期备份、顺序音视频流，而不适合作为通用交互式计算机的主力存储。

不要只用带宽评价存储设备

存储设备的性能至少要拆成顺序带宽、随机访问延迟、并发能力和写入放大。磁带的顺序带宽可以很高，但随机读写延迟足以让它无法承担现代文件系统的常规工作负载。

2.2 磁鼓：牺牲容量换定位延迟

为了改善一维磁带的随机访问，可以把带子“绕成一个硬的圆柱”，让圆柱高速旋转，并在不同轨道上放置读写头。这样，最坏访问延迟不再取决于整卷磁带要转多久，而主要取决于圆柱转一圈的时间。这就是磁鼓的设计思路。

²视频画面时间区间：00:10:05–00:10:10。

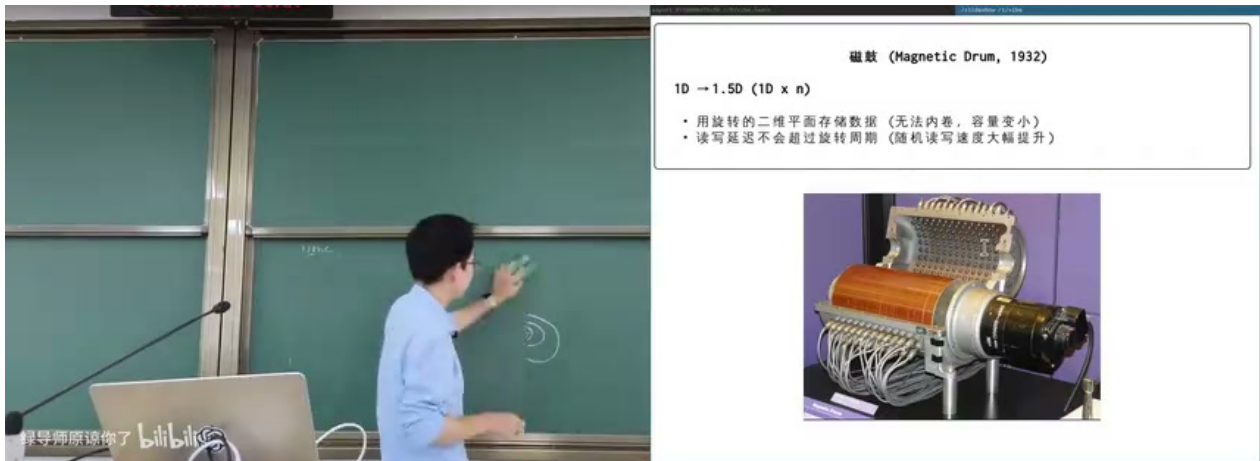


图 3: 磁鼓把一维磁介质组织成旋转圆柱，并用多个读写头并行降低定位等待时间。³

磁鼓从“线性移动”变成“周期性等待”，可以说是从 1D 向 1.5D 的过渡。容量比长磁带小，但随机访问延迟可控。这一思路继续向二维平面扩展，就得到磁盘：把磁道铺在圆形盘片上，用盘片旋转提供顺序流，用磁头移动切换磁道。

2.3 硬盘：旋转、寻道和工程极限

硬盘把一圈圈磁道布置在盘片上。读写头有两个关键动作：等待目标扇区旋转到读写头下面，移动读写头到目标磁道。于是一次随机读写的延迟大致由寻道时间、旋转等待和数据传输时间组成。

$$T_{io} \approx T_{seek} + T_{rotation} + T_{transfer}$$

其中， T_{seek} 是移动磁头到目标磁道的时间； $T_{rotation}$ 是等待目标扇区旋转到磁头下的时间； $T_{transfer}$ 是连续传输数据的时间。顺序读写可以把前两项摊薄，随机读写则会反复支付前两项。

³视频画面时间区间：00:17:25–00:17:30。

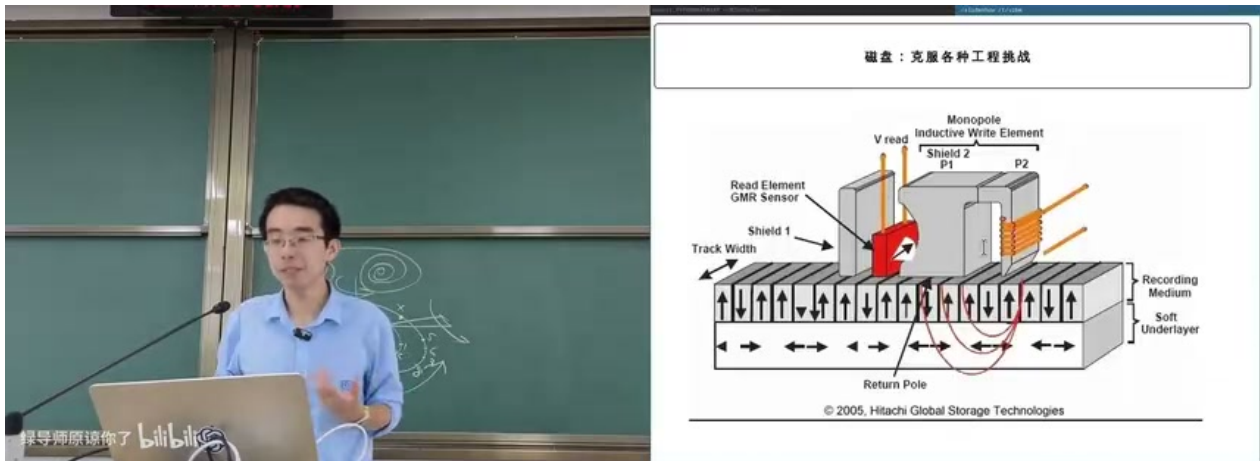


图 4: 硬盘读写头与磁畴的相互作用：读写头、磁场、磁畴方向和记录介质共同决定一个 bit 如何被读写。⁴

机械硬盘的工程成就来自多层“内卷”：更小的磁畴、更精密的材料、更高的转速、更多盘片、更好的读写头、更复杂的控制器。讲者也提醒，硬盘不是裸介质，而是一个带控制器的小系统；它会处理请求排序、缓存、错误校验、坏道映射等问题。

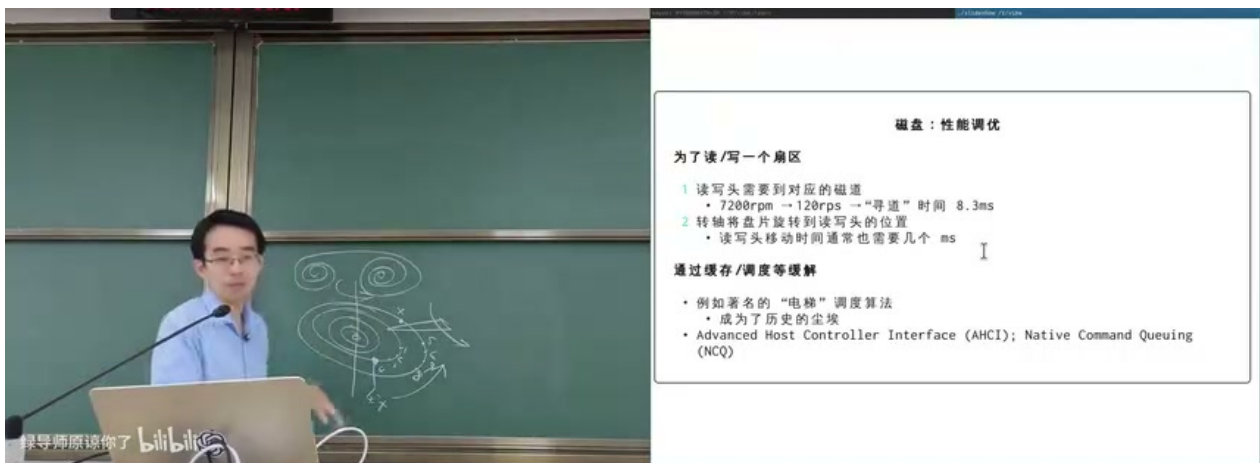


图 5: 硬盘性能调优的关键是减少寻道和旋转等待：顺序访问、调度算法、队列与缓存都会影响最终吞吐。⁵

为什么顺序访问能拯救硬盘

如果应用按连续块访问，磁头定位一次后，盘片旋转会自然把后续数据送到读写头下方，机械延迟被大量数据摊薄。如果应用随机访问大量小块，每个 I/O 都重新支付寻道和旋转等待，吞吐会急剧下降。

⁴视频画面时间区间：00:21:35–00:21:40。

⁵视频画面时间区间：00:26:55–00:27:00。

2.4 本章小结

磁存储从磁带到磁鼓再到硬盘，背后是同一个取舍：容量、成本、可靠性、随机访问延迟不可能同时免费。磁带赢在容量和长期归档；磁鼓用结构换延迟；硬盘把二维平面、机械定位和控制器组合起来，成为长期主流的块存储设备。

3 可移动介质：软盘与光盘

软盘和光盘都很适合展示“介质”和“驱动器”的分离。硬盘把盘片、读写头、控制器封在一个设备里；软盘和光盘则把低成本介质交给外部驱动器读取。这种设计降低了介质成本，也限制了速度、可靠性和密度。

3.1 软盘：低成本介质与数据交换文化

软盘的盘片相对裸露，机械结构松散，密度和性能都远不如硬盘。但在网络不普及时，它曾是个人之间传递数据、安装软件、发布游戏更新的重要媒介。软盘的意义不只在技术，也在它定义了一个时代的使用方式：把数据装进物理介质，带到另一台机器上。



图 6: 软盘曾是软件发行和数据交换的主力介质，低成本、可携带，但容量、速度和可靠性都受限。⁶

讲者提到，软盘虽然退出历史舞台，却以“保存”按钮图标形式留在软件界面中。这个细节很适合提醒我们：系统抽象会比设备本身活得更久。用户今天点击保存图标时，已经不需要知道软盘是什么，但“把易失状态变成持久状态”的语义仍然保留。

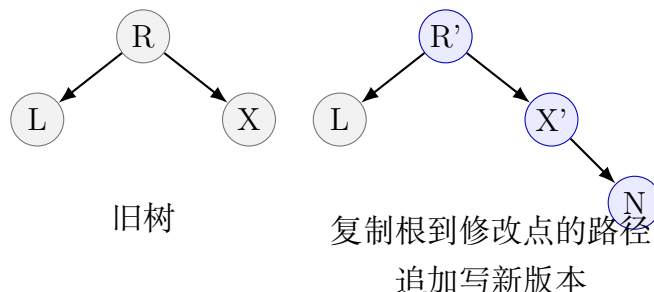
3.2 光盘：把坑做成数字内容

光盘采用“挖坑”存储：在透明塑料基底上形成极小的 pit/land 结构，用激光照射后的反射或干涉差异读取信息。它与磁盘一样旋转，也有随机访问和机械速度限制；但它有一个巨大的产业优势：复制便宜。一旦母盘做好，压盘可以在很短时间内复制大量内容。

⁶视频画面时间区间：00:31:35–00:31:40。

持久化数据结构

在数据结构语境中，“persistent”表示历史版本仍然可访问。它和存储设备里的 persistence 含义不同，但两者在 append-only 介质上相遇：为了在不可原地修改的介质上实现可变抽象，需要把修改变成追加写和新版本指针。



4.2 Project Silica: 在玻璃里写入长期数据

Project Silica 使用激光在玻璃中写入微结构，再用显微成像和解码读取。它不像传统磁盘或 SSD 那样强调通用随机读写，而是面向长期、低能耗、低维护的数据保存。其逻辑非常接近前面的抽象：random read 加 append-only write 足以承载丰富的数据结构。



图 8: Project Silica 将“挖坑”推进到玻璃体内部：长期保存、随机读取、追加写入，适合冷数据归档。⁸

这一段的的教学价值在于，它把硬件介质限制和软件数据结构联系起来。硬件给出“不能原地改”的约束，软件通过版本化、索引、追加日志和垃圾回收构造出看似可变的抽象。后面 SSD 的 FTL 也会再次出现同样思想。

4.3 本章小结

只追加写不是缺陷，而是一类强约束。它牺牲原地覆盖，换来介质简单性、历史可追溯性和长期可靠性。软件系统常常用日志、copy-on-write、持久化数据结构把这种约束转化

⁸ 视频画面时间区间：00:46:40–00:46:45。

成可用接口。

5 Flash 与 SSD：电路速度下的持久化

磁盘和光盘都要机械定位；Project Silica 也要复杂光学定位。要匹配现代 CPU 和内存系统的速度，最自然的方向是把存储本身做成电路。Flash memory 就是这样一种介质：用 floating gate 中的电荷状态保存信息。

5.1 Floating Gate：用电荷保存 bit

Flash cell 与 MOSFET 类似，但多了浮栅。通过较高电压把电子注入浮栅或释放出来，可以改变晶体管阈值电压，从而区分不同状态。SLC 每个 cell 存 1 bit，MLC/TLC/QLC 通过更多电压窗口在一个 cell 中存多个 bit，容量提高，可靠性和寿命下降。

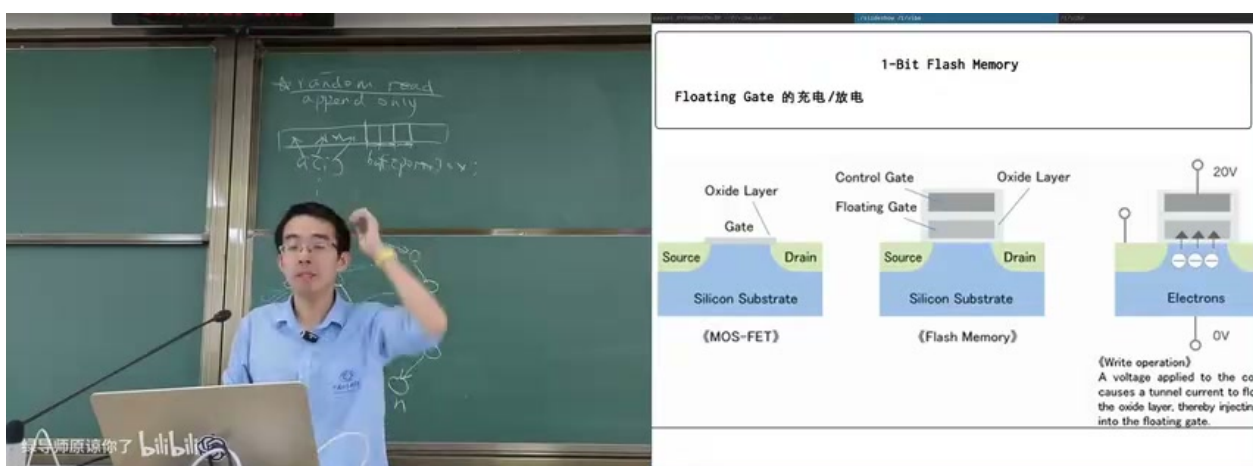


图 9: 1-bit Flash Memory：浮栅充放电改变阈值电压，把电荷状态变成可读取的数字状态。

9

Flash 的优势非常强：无机械部件，读写并行度高，颗粒可以横向扩展，体积小，抗震动，功耗低。消费级设备和数据中心逐渐从机械硬盘转向 SSD，根本原因不是“SSD 更时髦”，而是电路介质能提供机械设备很难达到的随机访问能力和并行带宽。

5.2 磨损、映射表和 FTL

Flash 的致命限制是擦写寿命。一次 erase 不可能完全恢复物理状态，经过几千到几次擦写后 cell 会磨损。单纯把逻辑块固定映射到物理块会导致热点块很快坏掉。因此 SSD 不是“裸 Flash 颗粒”，而是一个带 CPU、DRAM、控制器和固件的小型计算机系统。

⁹视频画面时间区间：00:55:35–00:55:40。

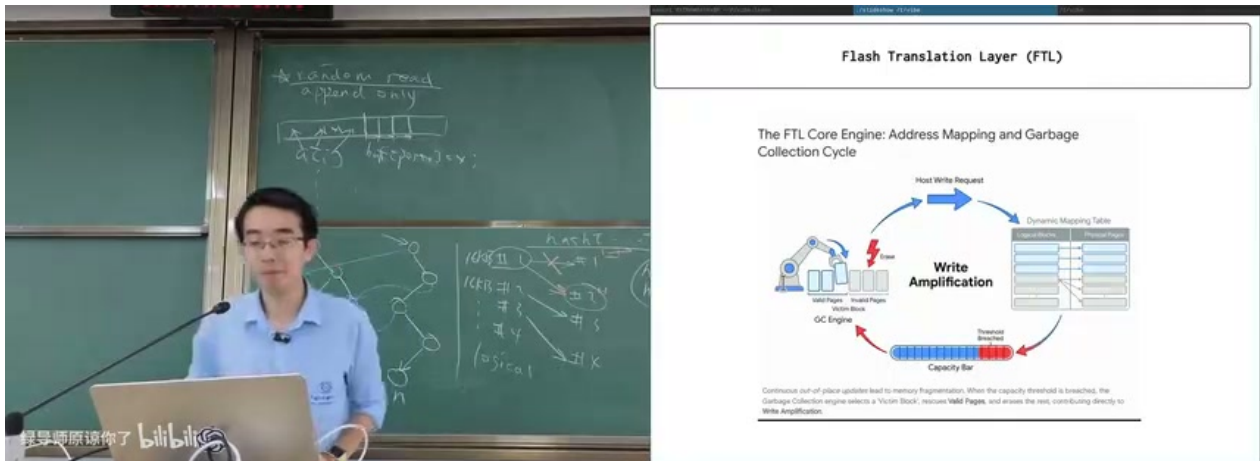


图 10: FTL 维护逻辑地址到物理地址的映射，并通过磨损均衡、垃圾回收和写放大控制 Flash 的寿命与性能。¹⁰

FTL (Flash Translation Layer) 的基本任务是维护 logical-to-physical 映射。操作系统以为自己在覆盖写逻辑块 1, FTL 可以把新数据写到另一个空闲物理页，再更新映射表。旧物理页变成无效页，之后由垃圾回收整理。这样，写入可以分散到不同物理块，实现 wear leveling。

```

1 write(logical_page, data):
2     physical_page = allocate_fresh_page()
3     program(physical_page, data)
4     old = map[logical_page]
5     map[logical_page] = physical_page
6     mark_invalid(old)
7     maybe_garbage_collect()

```

Listing 1: FTL 的简化逻辑：覆盖写被改写为追加写和映射更新

便宜 U 盘和假容量设备

存储设备会通过协议报告容量、型号和能力。控制器可以偷工减料，也可以伪造容量。过度便宜的 U 盘或 SSD 可能用小容量颗粒伪装成大容量设备，前面的写入看似成功，超过真实容量后数据直接丢失。

5.3 本章小结

Flash 把持久化推进到电路层，解决了机械随机访问的瓶颈；但它把复杂性转移到控制器和固件中。SSD 的高性能来自并行颗粒、缓存、映射表、垃圾回收和磨损均衡，而不是简单的“电子比机械快”。

¹⁰视频画面时间区间：01:05:20–01:05:25。

6 块设备：操作系统看到的统一形状

讲者在最后把所有介质拉回 OS 抽象：无论底层是磁盘、软盘、SSD、SD 卡还是 Project Silica，操作系统都需要一个可管理的接口。直接暴露 bit 或 byte 的成本太高；设备通常把数据组织成块。

6.1 为什么是 block array

定位是昂贵资源。对硬盘，定位意味着磁头、转轴和旋转等待；对光盘，定位意味着光学头和盘片旋转；对 SSD，定位意味着行列选择、电路面积、page/block 管理。为了让多个 byte 共享一次定位成本，设备以块为单位读写。

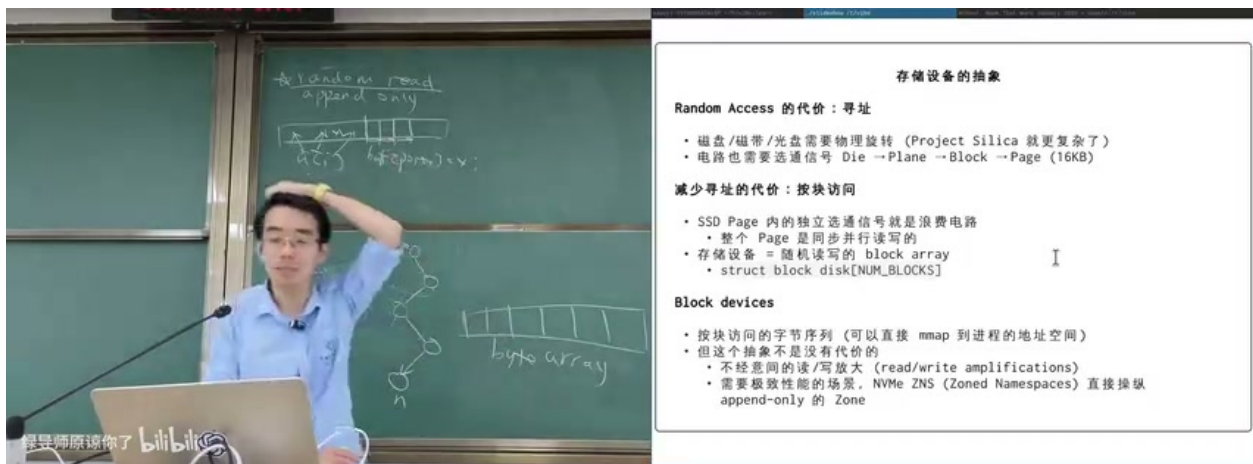


图 11: 存储设备抽象的结论：随机访问的代价来自寻址，设备用 block/page/sector 分摊定位和控制成本。¹¹

SSD 内部常见层次可以概括为 die、plane、block、page。读写通常按 page，擦除按更大的 block。磁盘和软盘常以 512 字节扇区为基本单位。操作系统在其上构造 buffer cache、page cache、文件系统、日志和 mmap，最终给用户进程一个更友好的字节级接口。

块设备接口

块设备的核心操作是：读一块、写一块、连续读写若干块。Linux 中相邻的抽象会把块设备 I/O、页缓存、文件系统和系统调用串起来，使普通程序可以用 read、write、mmap 访问复杂设备。

6.2 读放大、写放大和局部性

如果程序只改一个字节，设备可能必须读出 4KB、16KB 甚至更大的页，修改后再写回；如果 SSD 的有效页和无效页混在同一个擦除块中，垃圾回收还会搬移额外数据。这些都是放大效应。

¹¹ 视频画面时间区间：01:09:25–01:09:30。

$$A_w = \frac{\text{device bytes written}}{\text{user bytes requested}}$$

其中， A_w 是写放大系数；分子是设备实际写入、搬移或重写的字节数；分母是上层用户或文件系统原本请求写入的字节数。 $A_w > 1$ 表示底层为了维护介质约束付出了额外写入。

这也是为什么局部性重要。顺序写、大块写、对齐写、批量提交，都能让上层访问模式更符合设备的自然单位。随机小写则常常触发读改写、擦除、垃圾回收、队列拥塞和缓存失效。

6.3 把 OS 原理做成可观察工具

最后，讲者展示了一个由 AI 辅助写出的可视化工具：通过观察系统中的块 I/O，把读、写、同步、延迟分布、进程 I/O 统计展示出来。这个演示不是为了炫技，而是为了说明操作系统概念是可观察、可调试、可实验的。

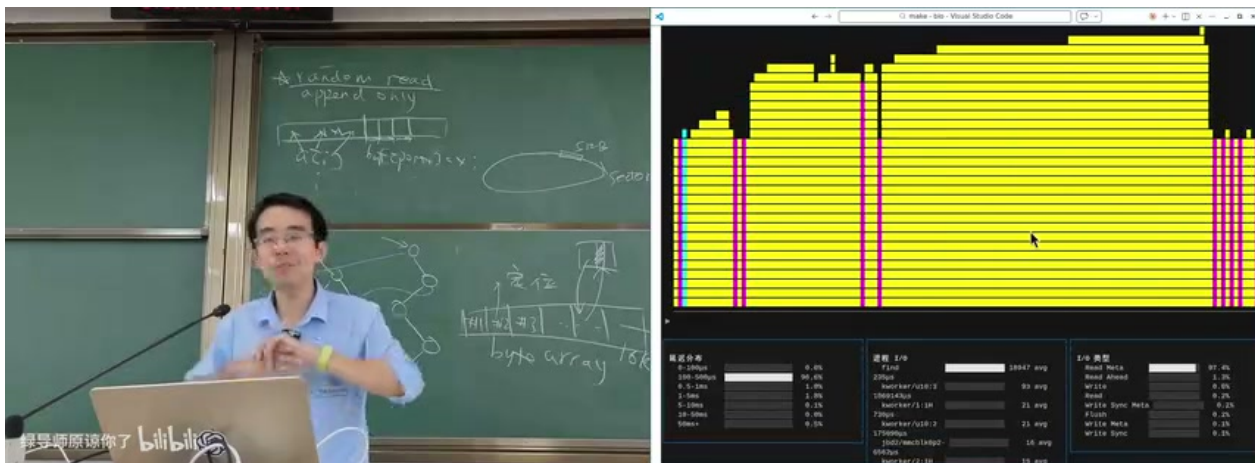


图 12: 块 I/O 可视化工具把系统中的读写请求变成动态图形，帮助理解文件访问最终落到块设备操作。¹²

```

1 # 顺序写：更容易形成大块连续 I/O
2 dd if=/dev/zero of=a.bin bs=1M count=100
3
4 # 遍历文件树：触发大量元数据和数据块读取
5 find / -maxdepth 3 -type f

```

Listing 2: 块设备视角下的访问模式实验

6.4 本章小结

块设备抽象是硬件成本和软件便利之间的妥协。硬件不愿意为每个 byte 付出独立定位成本，软件又希望看到连续地址空间。块、缓存、文件系统、页表、mmap、FTL 和调度器

¹²视频画面时间区间：01:18:40–01:18:45。

共同把这两个世界接起来。

7 总结与延伸

本讲的真正结论可以压缩成一句话：存储设备不是“保存数据的黑盒”，而是物理状态、定位机制、控制器固件和操作系统抽象共同构成的系统。一个 bit 可以是磁化方向、坑、电荷、玻璃中的微结构；一组 bit 可以按带、鼓、盘、页、块、扇区组织；而 OS 最终把它们尽量统一成块设备，再向上提供文件和内存映射。

讲者的收束也很有意思：在 AI 工具时代，操作系统知识的价值不只是背接口或记参数，而是帮助你提出可实验的问题。知道“块设备最终读写 block”，你就能想到用 eBPF 或其他机制观测 block I/O；知道“Flash 有擦写寿命”，你就能追问 FTL、磨损均衡和假容量设备；知道“随机访问需要定位”，你就能解释为什么顺序写和局部性仍然重要。

本讲带走的三条主线

1. 物理状态：持久化来自稳定、可读写、可区分的状态。
2. 定位代价：随机访问不是抽象免费赠品，而是硬件用电路、机械或光学系统买来的能力。
3. 软件补偿：FTL、文件系统、缓存、调度、持久化数据结构都在弥补介质限制。

进一步学习时，可以沿三条路径深入：第一，学习块设备层和 I/O 调度，理解 Linux 如何把 bio/request 提交给驱动；第二，学习文件系统日志、copy-on-write 和崩溃一致性，理解“写入成功”到底意味着什么；第三，学习 SSD 内部结构和 FTL，理解为什么同样容量的存储设备会有巨大性能和可靠性差异。